

JavaScript – prøv det ut selv

Konseptutvikling og programmering – du skriver koden på PC-en, og svarene på dette arket

Navn:

I dag skal du **ikke** følge med på tavla. Du skal prøve ting selv, og se hva som skjer.

Hver oppgave er bygd likt: du får litt kode, du skriver det inn i en fil, du kjører filen med Node, og så skriver du på dette arket hva som skjedde. Flere av oppgavene har **to** kodebiter, A og B, som er nesten like. Jobben din er å finne ut hva den lille forskjellen gjør.

Du trenger ikke huske noe utenat i dag. Du skal skrive kode og finne ut hva som skjer!

Jobb sammen, og spør når dere står fast :)

Ikke slett noe kode i dag: Lag en ny fil for hver oppgave og la de gamle ligge. Over hver oppgave står det hva fila skal hete.

Når timen er over, så har du en mappe full av små programmer som virker. Det er ditt eget bibliotek: står du fast senere, kan du åpne en fil du allerede har fått til, og se hvordan du gjorde det. Du kan også bygge videre på biblioteket ditt senere! Ingenting av det du skriver i dag er bortkastet.

Kom i gang:

1. Lag en mappe som heter `js` på skrivebordet.
2. Åpne VS Code. Velg **File** → **Open Folder**, og velg mappa `js`.
3. Lag en ny fil med **File** → **New File**, og gi den navnet som står over oppgaven.
4. Skriv koden i fila, og **lagre** med `Ctrl + S` (`Cmd + S` på Mac).
5. Åpne terminalen: **View** → **Terminal**.
6. Kjør fila med `node` og filnavnet, for eksempel `node 01-hei.js`

To ting du kommer til å trenge hele timen

1. Lagre før du kjører. `Ctrl + S`, eller `Cmd + S` på Mac.

`node` leser fila slik den ligger lagret på disken – ikke slik den ser ut på skjermen din. Endrer du koden uten å lagre, kjører maskinen den **gamle** versjonen, og du blir sittende og lure på hvorfor ingenting forandrer seg. Dette er den vanligste tidstyven av alle.

2. Trykk pil opp i terminalen. Du trenger ikke skrive `node 01-hei.js` om igjen hver eneste gang.

Klikk i terminalen, trykk **pil opp**, og forrige kommando dukker opp ferdig skrevet. Trykk **Enter**, så kjører den. Trykker du pil opp flere ganger, blar du lenger tilbake i det du har kjørt før.

Sløyfa du kommer til å gjenta hundre ganger i dag:

endre koden → `Ctrl + S` → klikk i terminalen → pil opp → **Enter**

Del 1 – console.log – å få maskinen til å si noe

En datamaskin gjør ikke noe av seg selv for å vise deg hva den driver med. Den regner ut og regner ut i stillhet.

`console.log( )` er beskjeden om at **dette vil jeg se!** Alt du putter inni parenteser, skriver maskinen ut i terminalen. Uten `console.log` så skjer det fremdeles ting, du får bare ikke vite hva.

1. Din første fil

Lag fila: 01-hei.js

```
console.log("Hei, verden!");  
console.log("Dette er min første fil.");
```

Skriv av det som kom i terminalen:

Hvor mange linjer kom det ut, og i hvilken rekkefølge?

Legg til en tredje `console.log`-linje med noe du finner på selv. Hvor i utskriften havnet den – øverst, i midten eller nederst? Hva forteller det deg om hvordan maskinen leser fila?

2. Hva skjer uten console.log?

Lag filene: 02a-stille.js og 02b-med-utskrift.js

A

```
2 + 3;
```

A skrev ut:

B

```
console.log(2 + 3);
```

B skrev ut:

Maskinen regnet ut `2 + 3` i **begge** filene. Så hva er det egentlig `console.log` gjør?

Dette er verdt å ta med seg: Når du senere sitter og lurere på hva programmet ditt driver med, er svaret nesten alltid å putte inn en `console.log` og se etter. Det er ikke juks – det er sånn alle programmerere jobber.

Kort oppsummert

- `console.log( )` er den eneste måten programmet snakker til deg på.
- Alt du putter inni parenteser, havner i terminalen.
- Maskinen leser fila ovenfra og ned, og skriver ut i den rekkefølgen.
- Står du fast senere: putt inn en `console.log` og se hva som faktisk ligger der.

Del 2 – Variabler

En variabel er en boks du legger en verdi i, og gir et navn. Da kan du bruke verdien senere, og bytte den ut når du vil.

3. Lag din første variabel

Lag fila: 03-navn.js

```
let navn = "Ada";  
console.log(navn);
```

Skriv av det som kom i terminalen:

Bytt nå ut `"Ada"` med ditt eget navn. Lagre. Kjør igjen.

Er `navn` et ord maskinen kjenner fra før, eller noe du fant på selv? Hva tror du hadde skjedd om du kalte variabelen `banan` i stedet?

4. Hermetegn eller ikke?

Lag filene: 04a-med-hermetegn.js og 04b-uten-hermetegn.js

A

```
let navn = "Ada";  
console.log("navn");
```

A skrev ut:

B

```
let navn = "Ada";  
console.log(navn);
```

B skrev ut:

Med hermetegn rundt fikk du noe helt annet enn uten. Hva er forskjellen på **"navn"** og **navn**?

5. En variabel kan bytte verdi

Lag fila: 05-poeng.js

```
let poeng = 0;  
console.log(poeng);  
poeng = 10;  
console.log(poeng);
```

Skriv av det som kom i terminalen:

Linja **console.log(poeng)** står helt likt begge steder. Hvorfor kom det likevel to forskjellige tall ut?

6. Forskjellen på let og const

Lag filene: 06a-let.js og 06b-const.js

Kjør A først, så B. Den ene virker. Den andre krasjer.

A

```
let pris = 25;
pris = 30;
console.log(pris);
```

A skrev ut:

B

```
const pris = 25;
pris = 30;
console.log(pris);
```

B skrev ut:

Skriv med egne ord: hva er forskjellen på **let** og **const** ?

Om feilmeldinger: Feilmeldinger er ikke kjeft. De er maskinen som forteller deg hva den ikke fikk til. Den **siste** linja er nesten alltid den viktige – resten er bare adressen til hvor det skjedde.

7. Er det et tall, eller er det tekst?

Lag filene: 07a-tall.js og 07b-tekst.js

De to kodebitene er like, bortsett fra to hermetegn.

A

```
let a = 2;
let b = 3;
let c = a + b;

console.log(c);
console.log(typeof a);
console.log(typeof c);
```

A skrev ut:

B

```
let a = "2";
let b = 3;
let c = a + b;

console.log(c);
console.log(typeof a);
console.log(typeof c);
```

B skrev ut:

+ gjorde to helt forskjellige ting i A og B. Hva gjorde den i hver av dem?

`typeof` forteller hva slags verdi maskinen tror den har. Hva kalte den de to variantene av `a`?

`c` er svaret på regnestykket, lagret i sin egen variabel. Hva slags type ble `c` i A, og hva ble den i B? I B var bare **én** av verdiene tekst – og likevel ble hele svaret tekst. Hvorfor, tror du?

8. To måter å sette sammen tekst på

Lag filene: 08a-pluss.js og 08b-dollar.js

A

```
let navn = "Ada";
console.log("Hei " + navn + "!");
```

A skrev ut:

B

```
let navn = "Ada";
console.log(`Hei ${navn}!`);
```

B skrev ut:

Ble resultatet likt eller ulikt? Hvilken av dem synes du er lettest å lese?

Om tegnet i B: Tegnet rundt teksten i B er ikke et vanlig hermetegn, men en **backtick**: ```. På norsk tastatur ligger den til venstre for backspace, og du må ofte trykke den **to ganger** for at den skal dukke opp. Inne i en slik tekst kan du putte en variabel rett inn med `${ }`.

Kort oppsummert

- `let` lager en variabel du kan endre. `const` kan ikke endres etterpå.
- Hermetegn avgjør alt: `"navn"` er selve ordet, `navn` er innholdet i variabelen.
- `2` er et tall, `"2"` er tekst. `+` legger sammen tall, men limer sammen tekst – og er én av delene tekst, blir hele svaret tekst.
- `typeof` forteller deg hva maskinen tror den har.

Del 3 – if og else

Her lærer du koden å velge vei: gjør én ting hvis noe stemmer, og noe annet hvis det ikke gjør det.

9. Samme kode, tre forskjellige svar

Lag fila: 09-aldersgrense.js

```
let alder = 15;

if (alder >= 16) {
  console.log("Du får kjøpe energidrikk.");
} else {
  console.log("Beklager, 16-årsgrense.");
}
```

Kjør den tre ganger. Endre **alder** mellom hver gang, og skriv ned hva som kom ut:

alder = 15

alder = 16

alder = 17

Koden var helt lik alle tre gangene. Hva var det som bestemte hvilken linje som ble skrevet ut?

10. Forskjellen på > og >=

Lag filene: 10a-storre.js og 10b-storre-lik.js

Her er **alder 16 i begge**. Det eneste som skiller dem, er ett tegn.

A

```
let alder = 16;

if (alder > 16) {
  console.log("Du får kjøpe.");
} else {
  console.log("For ung.");
}
```

A skrev ut:

B

```
let alder = 16;

if (alder >= 16) {
  console.log("Du får kjøpe.");
} else {
  console.log("For ung.");
}
```

B skrev ut:

Hva betyr **>=**, og hvorfor ga det et annet svar enn **>**?

11. Den klassiske feilen: = og ===

Lag filene: 11a-tre-likhetstegn.js og 11b-ett-likhetstegn.js

Denne oppgaven viser en feil nesten alle gjør. Se godt på antall likhetstegn.

A

```
let alder = 20;

if (alder === 16) {
  console.log("Du er 16.");
}
console.log(alder);
```

A skrev ut:

B

```
let alder = 20;

if (alder = 16) {
  console.log("Du er 16.");
}
console.log(alder);
```

B skrev ut:

I B ble det skrevet ut at du er 16, selv om alderen var 20. Og på siste linje hadde **alder** forandret seg. Hva tror du **=** gjorde inne i if-setningen?

Husk: **=** setter en verdi. **===** sammenligner to verdier. Skriver du **=** der du mente **===**, krasjer ingenting – programmet gjør bare noe helt annet enn du trodde. Det er derfor denne feilen er så vanskelig å finne.

12. Rekkefølgen betyr noe

Lag fila: 12-forerkort.js

Denne skal fortelle hva slags førerkort du kan ta. Du er 20 år.

```
let alder = 20;

if (alder >= 16) {
  console.log("Du kan ta moped.");
} else if (alder >= 18) {
  console.log("Du kan ta bil.");
}
```

Skriv av det som kom i terminalen:

Du er 20 år og burde fått bil. Hvorfor sa den moped?

Hvordan ville du rettet koden? Skriv forslaget ditt her:

Kort oppsummert

- `if` velger vei. `else` er det som skjer ellers.
- `>=` tar med tallet selv. `>` gjør det ikke.
- `=` setter en verdi, `===` sammenligner. Dette er den vanligste feilen av alle.
- Med `else if` vinner den **første** som stemmer – derfor betyr rekkefølgen noe.

Del 4 – Løkker

En løkke gjentar den samme jobben mange ganger, uten at du må skrive den mange ganger.

13. Gjett først, kjør etterpå

Lag fila: 13-telle.js

Ikke kjør denne ennå. Les den, og skriv først ned hva du **tror** kommer ut:

```
for (let i = 0; i < 5; i++) {  
  console.log(i);  
}
```

Jeg tror det kommer:

Det kom faktisk:

En **for**-løkke har tre deler inne i parentesene, skilt med semikolon:

```
for (let i = 0 ; i < 5 ; i++ ) {  
  //      start      stopp når      etter  
  //      her        dette ikke     hver  
  //                        er sant      runde
```

Hvor mange linjer ble skrevet ut, og hvilket tall stod på den siste?

14. Forskjellen på < og <=

Lag filene: 14a-mindre-enn.js og 14b-mindre-lik.js

A

```
for (let i = 0; i < 5; i++) {  
  console.log(i);  
}
```

A skrev ut:

B

```
for (let i = 0; i <= 5; i++) {  
  console.log(i);  
}
```

B skrev ut:

Hvor mange linjer ga hver av dem?

Endre A slik at den skriver ut tallene 1, 2, 3, 4 og 5 – og ingenting annet. Skriv av løkka du endte opp med:

15. for og while gjør den samme jobben

Lag filene: 15a-for.js og 15b-while.js

A

```
for (let i = 1; i <= 3; i++) {  
  console.log(i);  
}
```

A skrev ut:

B

```
let i = 1;  
while (i <= 3) {  
  console.log(i);  
  i = i + 1;  
}
```

B skrev ut:

De tre delene **start**, **stopp når** og **etter hver runde** finnes i begge. I A står de på én linje. Finn dem i B, og skriv opp hvilke linjer de står på:

16. Løkka som aldri stopper

Lag fila: 16-uendelig.js

Nå skal du lage en feil med vilje. **Les hele oppgaven før du kjører.**

Åpne `15b-while.js`, marker alt, og kopier. Lag den nye fila, lim inn – og slett linja `i = i + 1;`. Slik beholder du den som virker.

Lagre og kjør. Programmet stopper aldri av seg selv. **Stopp det med `Ctrl + C` i terminalen.** Dette gjelder også på Mac – ikke `Cmd`.

Hvorfor stoppet den ikke? Hva var det som ikke skjedde?

Dette kommer til å skje deg igjen: En `while`-løkke må alltid ha noe inni seg som til slutt gjør betingelsen usann. Glemmer du det, går den for alltid. Når terminalen «henger» og bare spyr ut tekst, er `Ctrl + C` svaret.

Kort oppsummert

- En løkke gjentar den samme jobben, uten at du må skrive den mange ganger.
- `for` har start, stopp-når og etter-hver-runde samlet på én linje.
- `while` har nøyaktig de samme tre delene, bare spredt utover.
- Glemmer du opptellingen, stopper løkka aldri. Da er `Ctrl + C` svaret.

Del 5 – Lister

En liste er én variabel som holder på mange verdier samtidig – som en rad med nummererte bokser.

17. Hente ting ut av en liste

Lag fila: 17-liste.js

```
let dyr = ["hund", "katt", "hest"];

console.log(dyr[0]);
console.log(dyr[1]);
console.log(dyr.length);
console.log(dyr[3]);
```

Skriv av det som kom i terminalen:

Hvilket nummer har `"hest"` ?

Lista har tre ting i seg. Hvorfor er ikke `"hund"` nummer 1?

Den siste linja ga `undefined` . Hva tror du det betyr?

18. Forskjellen på `i` og `dyr[i]`

Lag filene: 18a-nummer.js og 18b-verdi.js

Dette er den vanligste forvekslingen når man begynner med lister. Begge løkkene går like mange runder.

A

```
let dyr = ["hund", "katt", "hest"];

for (let i = 0; i < dyr.length; i++) {
  console.log(i);
}
```

B

```
let dyr = ["hund", "katt", "hest"];

for (let i = 0; i < dyr.length; i++) {
  console.log(dyr[i]);
}
```

A skrev ut:

B skrev ut:

Med egne ord: hva er `i` , og hva er `dyr[i]` ?

19. Løkke og if sammen

Lag fila: 19-tell.js

Her er en liste med tall:

```
let tall = [4, 9, 2, 7, 12];
```

Skriv et program som teller hvor mange av tallene som er **større enn 5**, og skriver ut antallet.

Tips: Du trenger tre ting, i denne rekkefølgen:

1. en teller som starter på null, **utenfor** løkka: `let antall = 0;`
2. en løkke som går gjennom lista, som i forrige oppgave
3. en **if** **inne i** løkka, som legger 1 til telleren når tallet er stort nok

Til slutt skriver du ut telleren – utenfor løkka igjen.

Hvor mange tall var større enn 5?

Skriv av programmet ditt her:

Ekstra 🌟 – klarer du å få den til å skrive ut **hvilke** tall som var over 5, i tillegg til hvor mange?

Kort oppsummert

- En liste holder på mange verdier i én variabel.
- Tellingene starter på 0, så `dyr[0]` er den **første**.
- `.length` sier hvor mange elementer lista har.
- `i` er plassnummeret, `dyr[i]` er verdien som ligger der.

Biblioteket ditt

Kryss av for filene du rakk. Alt som er kryssset av, er et program du har fått til å virke.

- | | | |
|--|---|---|
| ☐ 01-hei.js | ☐ 02a-stille.js | ☐ 02b-med-utskrift.js |
| ☐ 03-navn.js | ☐ 04a-med-hermetegn.js | ☐ 04b-uten-hermetegn.js |
| ☐ 05-poeng.js | ☐ 06a-let.js | ☐ 06b-const.js |
| ☐ 07a-tall.js | ☐ 07b-tekst.js | ☐ 08a-pluss.js |
| ☐ 08b-dollar.js | ☐ 09-aldersgrense.js | ☐ 10a-storre.js |
| ☐ 10b-storre-lik.js | ☐ 11a-tre-likhetstegn.js | ☐ 11b-ett-likhetstegn.js |
| ☐ 12-forerkort.js | ☐ 13-telle.js | ☐ 14a-mindre-enn.js |
| ☐ 14b-mindre-lik.js | ☐ 15a-for.js | ☐ 15b-while.js |
| ☐ 16-uendelig.js | ☐ 17-liste.js | ☐ 18a-nummer.js |
| ☐ 18b-verdi.js | ☐ 19-tell.js | |

Ordliste – alt du møtte i dag

<code>console.log(x)</code>	Skriver ut <code>x</code> i terminalen. Uten denne ser du ingenting.
<code>node 01-hei.js</code>	Kjører fila. Husk å lagre først.
<code>let navn = "Ada";</code>	Lager en variabel du kan endre senere.
<code>const pris = 25;</code>	Lager en variabel som ikke kan endres. Prøver du, krasjer det.
<code>"navn"</code> mot <code>navn</code>	Med hermetegn: selve ordet. Uten: det som ligger i variabelen.
<code>+</code>	Legger sammen tall – men limer sammen tekst.
<code>typeof x</code>	Forteller om <code>x</code> er <code>number</code> eller <code>string</code> .
<code>=</code>	Setter en verdi.
<code>===</code>	Sammenligner to verdier. Det er denne du vil ha i en <code>if</code> .
<code>></code> og <code>>=</code>	Større enn / større enn eller lik.
<code>if (...) { }</code>	Gjør det inni krøllparentesene hvis det i parentesen stemmer.
<code>else { }</code>	Gjør dette i stedet, hvis det ikke stemte.
<code>else if (...)</code>	Sjekker noe nytt. Den første som stemmer, vinner.
<code>for (let i = 0; i < 5; i++)</code>	Gjentar noe et bestemt antall ganger.
<code>while (...)</code>	Gjentar så lenge noe stemmer. Må ha noe inni som til slutt stopper den.
<code>Ctrl + C</code>	Stopper et program som har låst seg. Også på Mac.
<code>Ctrl + S</code>	Lagrer fila. Må gjøres før du kjører.
Pil opp	Henter fram forrige kommando i terminalen, så du slipper å skrive den igjen.
<code>let dyr = ["hund", "katt"]</code>	Lager en liste.
<code>dyr[0]</code>	Henter det første elementet. Tellingen starter på null.
<code>dyr.length</code>	Hvor mange elementer lista har.